CASTILLO SERRATO ANGEL

323052214

Sobrecarga contra sobreescritura:

En el código, un ejemplo de sobrecarga ocurre en la clase *BandaTransportadora* con los métodos *avanzar()* y *avanzar(int metros),* donde el mismo nombre de método se utiliza dos veces cambiando sus parámetros. Un ejemplo de sobreescritura ocurre en la clase *Alarma* con el método *reaccionar(boolean alerta)* el cual lleva la anotación *@Override* para definir el comportamiento del método heredado de *Actuador*. La diferencia es que la sobrecarga se da dentro de la misma clase al cambiarle los parámetros, mientras que la sobreescritura se da entre una clase padre y una hija conservando la misma firma del método para adaptarle el comportamiento.

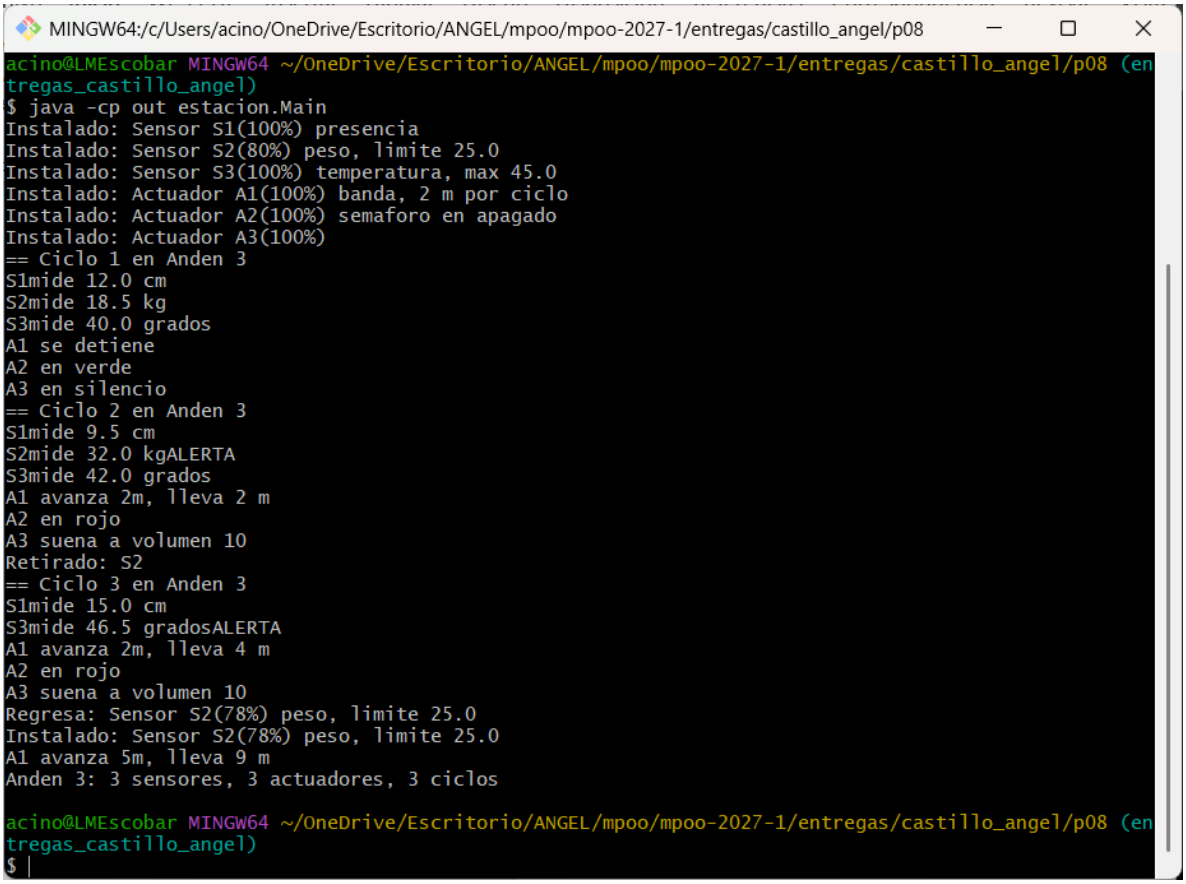
Composición contra agregación:

La *Bateria* representa una composición porque es instanciada directamente dentro de la clase *Dispositivo* mediante *new Bateria(carga),* esta ligada a lo que le pase a dispositivo. En cambio, los sensores en la *Estacion* es de agregación porque los sensores nacen en *Main* y se le entregan a la estación por medio del método *instalar().* Esto se ve en el *ciclo 3* cuando la báscula *S2* es retira de la estación con *anden.retirar("S2"):* el objeto no se destruye, mantiene sus datos en como su batería al 78% y se vuelve a instalar más adelante sin perder su estado.

Modificadores de acceso:

Al agregar la línea *bascula.gastar(5),* en Main se obtiene el error: *gastar(int) has protected access* in Dispositivo. Esto pasa porque el método gastar tiene el modificador de *protected* en la clase Dispositivo. El modificador *protected* solo nos permite que el método sea invocado por clases dentro del mismo paquete o por sus hijas. *SensorPeso* sí puede invocar *gastar* porque hereda de Sensor (esta a su vez hereda de Dispositivo), mientras que Main no puede llamarlo porque se encuentra en el paquete de *estacion* y no extiende de *Dispositivo.*

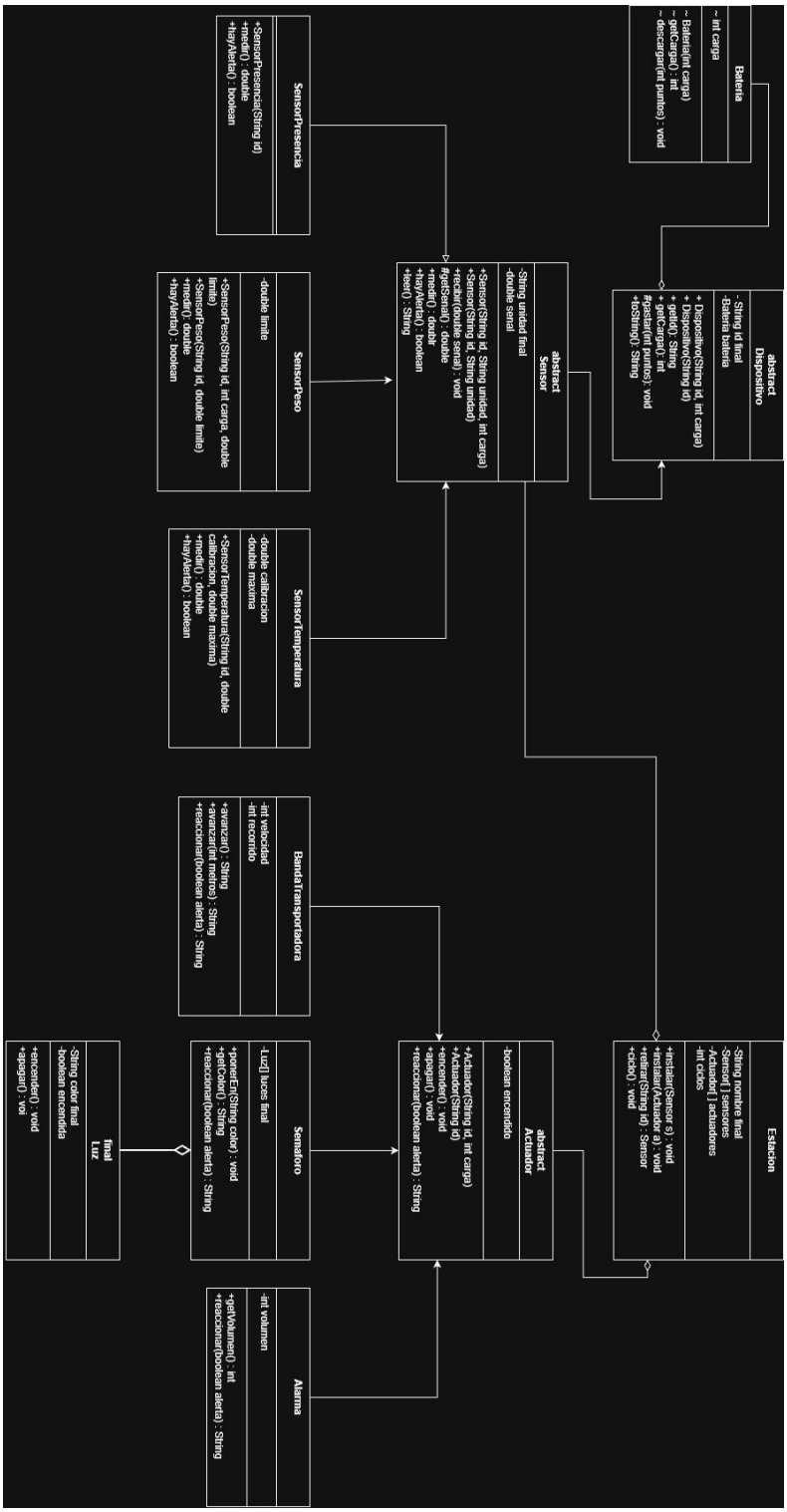
CAPTURA DE EJECUCION:



Tuve algunos percances al momento de compilar y ejecutar, puesto que en main venían unos valores enteros, por ejemplo para la senal, que tenían que ser double. A su vez me pareció adecuado utilizar setters para las validaciones en batería por ejemplo entre otras. He de decir que si fue considerablemente mas laboriosa esta practica pero muy disfrutable a mi parecer.

También me quede con una duda al momento de usar los setters, porque (por ejemplo) el método *setLimite* en *SensorPeso* funciona exactamente igual con o sin la palabra public?

DIAGRAMA:


Bateria
~ int carga
~ Bateria(int carga)
~ getCarga() : int
~ descargar(int puntos) : void
abstract
Dispositivo
- String id final
- Bateria bateria
+ Dispositivo(String id, int carga)
+ Dispositivo(String id)
+ getId() : String
+ getCarga() : int
#registar(int puntos) : void
+toString() : String
Estacion
-String nombre final
-Sensor[ ] sensores
-Actuador[ ] actuadores
-int ciclos
+instalar(Sensor s) : void
+instalar(Actuador a) : void
+retirar(String id) : Sensor
+ciclo() : void
SensorPresencia
+SensorPresencia(String id)
+medir() : double
+hayAlerta() : boolean
SensorPeso
-double limite
+SensorPeso(String id, int carga, double limite)
+SensorPeso(String id, double limite)
+medir() : double
+hayAlerta() : boolean
abstract
Sensor
-String unidad final
-double senal
+Sensor(String id, String unidad, int carga)
+Sensor(String id, String unidad)
+recibir(double senal) : void
#getSenal() : double
+medir() : double
+hayAlerta() : boolean
+toS() : String
SensorTemperatura
-double calibracion
-double maxima
+SensorTemperatura(String id, double calibracion, double maxima)
+medir() : double
+hayAlerta() : boolean
BandaTransportadora
-int velocidad
-int recorrido
+arrancar() : String
+arrancar(int metros) : String
+reaccionar(boolean alerta) : String
Semaforo
-Luz[] luces final
+ponerEn(String color) : void
+getColor() : String
+reaccionar(boolean alerta) : String
final
Luz
-String color final
-boolean encendida
+encender() : void
+apagar() : voi
abstract
Actuador
-boolean encendido
+Actuador(String id, int carga)
+Actuador(String id)
+encender() : void
+apagar() : void
+reaccionar(boolean alerta) : String
Alarma
-int volumen
+getVolumen() : int
+reaccionar(boolean alerta) : String